Software Requirements Specification (SRS)

Demand Forecasting and Restock Advisor for Small Shops (working title)

1. Title Page and Document Information

- Version: 0.1 (draft for team review)
- Date: 26 October 2026
- Status: Awaiting team approval. No design or code exists yet.
- Team: 4 third-year CSE students (names and roles: TBD)
- Context: Machine Learning evaluation project (1 credit). The college gives no SRS template, so a standard IEEE-style structure is used.
- Conventions: [E] = essential for the MVP, [O] = optional enhancement, TBD = unresolved decision listed in Appendix A.
- ID prefixes: FR functional, NFR non-functional, ML machine learning, BE backend/API, DB database, FE frontend, SEC security, AC acceptance.

2. Introduction and Project Overview

2.1 Purpose

This document states what the system must do, so the team agrees on scope before design starts. The Detailed Design Specification (DDS) follows only after this SRS is approved.

2.2 Overview

The system is a web application for small shop owners. The owner uploads a CSV file of past daily sales. A Java Spring Boot backend validates the file and stores it in MySQL. A Python ML component forecasts daily units sold per product for the next 7 to 14 days. The backend then converts the forecast into a restock suggestion using current stock and supplier lead time. Results are shown in the browser.

2.3 Related work (from the team's literature review)

- Azzahra and Sudirman (2025) compared a 7-day moving average, ARIMA and SARIMA on one coffee-shop product. ARIMA(0,0,1)(0,0,1)[7] had the lowest MAE and RMSE and SARIMA the highest. The data came from a single shop and was not public.

- Cherian et al. (2025) compared SARIMA and DeepAR and described a web interface where the user uploads a CSV and enters a horizon, store ID and SKU ID. Their single SKU/store demo reports 50.76% (SARIMA), 81.0% (DeepAR) and 77.14% (combined). The paper does not define 'accuracy', and the combined model scores below DeepAR alone, so these figures must not be compared with ours.
- Kim and Cho (2026) studied hybrid clustering for retail demand forecasting (citation details TBD).

This project adopts the upload-and-forecast workflow but uses lighter models than DeepAR.

3. Problem Statement

Small shops must decide how much of each product to reorder. Ordering too little causes stock-outs and lost sales. Ordering too much ties up money and, for perishable goods, causes waste. Both reference papers describe this problem for small businesses. The team assumes many owners decide mostly from experience and lack an accessible tool that turns their own sales history into a concrete reorder quantity (assumption not yet validated).

4. Project Objectives

- OBJ-01: Build a working end-to-end system: browser, Spring Boot API, Python ML component, MySQL.
- OBJ-02: Produce per-product daily demand forecasts from uploaded sales history.
- OBJ-03: Turn forecasts into explainable restock suggestions.
- OBJ-04: Compare at least one ML or statistical model against simple baselines on a chronological hold-out test, and report the real results, including the case where a baseline wins.
- OBJ-05: Keep SRS, DDS, schema, API and code consistent, and keep the project small enough to finish and explain.

5. Scope of the Project

In scope [E]: CSV upload and validation, MySQL storage, per-product forecast, restock suggestion, history view, downloadable CSV template, documentation to run and test.

In scope [O]: inventory/price/promotion columns used as extra inputs, CSV export, model comparison view, Indian festival calendar feature, delete an upload, user login.

Out of scope: deep learning (DeepAR, LSTM) unless later justified in writing; external AI APIs; real-time or POS integration; automatic ordering from suppliers; price optimisation; mobile app; cloud deployment; the final academic report (handled separately).

6. Target Users and Stakeholders

- Shop owner or manager (primary): non-technical, keeps sales records in Excel or an exported CSV, decides restocking.
- Evaluator/faculty: wants a working, explainable ML system.
- Development team (4 students): builds, tests and maintains it.

7. Functional Requirements

7.1 Data upload and validation

- FR-01 [E] The user shall upload a CSV file through a web form.
- FR-02 [E] The file shall contain at least: date, product identifier, units sold. Column names follow a documented template (flexible column mapping: TBD, D-04).
- FR-03 [E] The system shall validate file type, size limit, required columns, parseable dates and non-negative numeric units sold, and reject invalid files or rows with messages naming the column and row.
- FR-04 [E] The system shall detect duplicate (date, store, product) rows and apply one documented policy (TBD, D-05).
- FR-05 [E] The system shall apply one documented policy for days with no record for a product (TBD, D-05).
- FR-06 [E] Validated data shall be stored in MySQL linked to an upload record.
- FR-07 [O] Optional columns (store ID, category, inventory level, price, discount, promotion) shall be accepted, stored and used when present.
- FR-08 [E] The user shall be able to download a sample CSV template.

7.2 Forecasting

- FR-09 [E] The user shall choose an upload, one or more products and a horizon (7 or 14 days; default TBD).
- FR-10 [E] The system shall return a daily forecast of units sold for each selected product through the ML component.
- FR-11 [E] A forecast shall use only data dated before the forecast start (no future information).

- FR-12 [E] The forecast shall be shown as a table and a line chart together with recent history.
- FR-13 [E] The result shall show the model used and its MAE and RMSE from the last evaluation run.
- FR-14 [E] If a product has too little history (minimum TBD, D-06), the system shall say so and return no forecast.

7.3 Restock advisor

- FR-15 [E] The user shall provide or confirm current stock (taken from the latest inventory column if present, otherwise entered manually), lead time in days and a safety-stock setting.
- FR-16 [E] The system shall compute the suggested order quantity as max(0, forecast demand over the lead time + safety stock - current stock). Safety-stock method and defaults: TBD (D-03).
- FR-17 [E] The suggestion shall come with a plain-language explanation showing forecast demand, safety stock and current stock, and shall flag a stock-out risk when current stock is below forecast demand over the lead time.

7.4 History and management

- FR-18 [E] The user shall see a list of past uploads, forecasts and suggestions and open any of them.
- FR-19 [E] When the ML component or database is unavailable, the system shall show a clear error message instead of failing silently.
- FR-20 [O] Delete an upload and all its related data.
- FR-21 [O] Export suggestions as CSV.
- FR-22 [O] Compare baseline and candidate models in the UI.
- FR-23 [O] Use an Indian festival calendar as a model feature.
- FR-24 [O] User login with separate data per user (TBD, D-11).

8. Non-Functional Requirements

- NFR-01 Usability: a non-technical user shall be able to go from upload to suggestion using on-screen instructions and the template only.
- NFR-02 Performance (proposed, confirm after a prototype): validating and storing a file at the size limit within 30 seconds, and a single-product forecast within 10 seconds, on a standard student laptop.

- NFR-03 Reliability: invalid input shall never crash the backend; every error returns a structured error response.
- NFR-04 Maintainability: layered backend (controller, service, repository); ML code in a separate module; README for setup.
- NFR-05 Reproducibility: fixed random seeds, pinned library versions, training reproducible from scripts.
- NFR-06 Portability: runs locally with documented setup (operating system: TBD).
- NFR-07 Responsive and accessible UI: usable from phone-width to desktop screens, readable fonts and sufficient contrast.
- NFR-08 Honesty: every accuracy figure shown comes from an actual run, and synthetic data is labelled as synthetic.
- NFR-09 Logging: uploads, validation failures, forecast requests and errors are logged without storing sensitive content.
- NFR-10 Testability: every FR has at least one test case.

9. User Stories and Use Cases

9.1 User stories

- US-01: As an owner, I want to upload my sales file so that the system can learn my demand pattern.
- US-02: As an owner, I want to see next week's expected sales per product so that I can plan stock.
- US-03: As an owner, I want a suggested reorder quantity with the reason so that I can trust it.
- US-04: As an owner, I want to see past forecasts so that I can compare them with what actually sold.
- US-05: As an evaluator, I want to see model metrics against a baseline so that I can judge the ML work.

9.2 Use cases

- UC-01 Upload sales data. Actor: owner. Precondition: CSV in template format. Main flow: choose file, submit, system validates, stores and shows a summary. Alternate: validation fails, errors listed, nothing stored.
- UC-02 Generate forecast. Precondition: an upload exists. Main flow: select upload, products and horizon; backend calls the ML component; table and chart are shown. Alternate: too little history, message shown; ML component down, error shown.

- UC-03 Get restock suggestion. Precondition: forecast exists. Main flow: enter or confirm stock, lead time and safety stock; system shows suggested quantity and explanation.
- UC-04 View history. Main flow: open history list, select an item, view stored result.

10. System Constraints and Assumptions

Constraints

- Stack: Python (pandas, NumPy, scikit-learn, Matplotlib, Seaborn), Java Spring Boot, MySQL, HTML/CSS/JavaScript, VS Code, Git/GitHub.
- No deep learning and no external AI APIs unless later approved with justification.
- Runs locally on a student laptop; no cloud deployment.
- Team of 4 beginners in ML and full-stack development; the team manages its own timeline.
- The main dataset is synthetic.

Assumptions

- Uploaded data is daily and at product level.
- Lead time is one fixed number of days per product.
- Units sold approximates demand, but it can understate demand when an item was out of stock.
- Uploaded files contain no personal customer data.

11. Hardware and Software Requirements

(Versions marked proposed will be fixed in the DDS.)

- Development machine: laptop with at least 8 GB RAM (proposed).
- Editor: VS Code. Version control: Git and GitHub.
- Java: JDK 17 or later (proposed), Spring Boot 3.x (proposed), build tool Maven (proposed, TBD).
- Database: MySQL 8.x.
- Python: 3.10 or later (proposed) with pandas, NumPy, scikit-learn, Matplotlib, Seaborn. statsmodels only if SARIMA/ARIMA is chosen (D-02).
- Browser: current Chrome, Edge or Firefox.

12. Dataset Requirements and Expected Data Sources

12.1 Primary dataset

Kaggle dataset 'Retail Store Inventory and Demand Forecasting', described by its author as synthetically generated. Columns stated in its description: Date, Store ID, Product ID, Category, Region, Inventory Level, Units Sold, Units Ordered, Price, Discount, Weather Condition, Promotion, Competitor Pricing, Seasonality, Epidemic, Demand.

Status: not yet downloaded or inspected. Row count, date range, missing values, duplicates, ID consistency and licence are unknown (D-12) and will be recorded after inspection.

12.2 Known issues to handle

- The Demand column is an estimated value and shall not be used as a model input (leakage risk).
- Units Ordered and Inventory Level shall be checked for leakage before use as features.
- Weather, Competitor Pricing, Epidemic and Region will not exist in a real shop's upload, so the production model shall not depend on them.
- The data is generic, with no Indian festival effects unless we add them (FR-23).

12.3 Other sources

- User-uploaded CSV files (the real use case).
- Optional: anonymised sales export from a real local shop, with the owner's permission (D-07).

12.4 Data requirements

- Required fields: date, product identifier, units sold. Needed for restock advice: current stock.
- Train, validation and test sets shall be split chronologically, never randomly. Ratios: TBD.

13. Machine Learning Requirements

- ML-01 [E] Target: daily units sold per product (per store when a store column exists).
- ML-02 [E] Baselines: naive (same weekday last week) and 7-day moving average.
- ML-03 [E] At least one candidate beyond the baselines, chosen from scikit-learn regression on lag, rolling and calendar features, and optionally ARIMA/SARIMA (needs statsmodels). Final set: TBD (D-02). DeepAR and LSTM are out of scope.

- ML-04 [E] Features shall use only past information. The Demand column shall not be an input.
- ML-05 [E] Metrics: MAE and RMSE. MAPE or sMAPE only with documented handling of zero sales.
- ML-06 [E] Candidates shall be compared with the baselines on the held-out test period. Reported numbers come only from actual runs.
- ML-07 [E] Trained models and preprocessing shall be saved with metadata: training date, data range, metrics.
- ML-08 [E] A reusable prediction interface: input is a product's history and a horizon, output is dates, predicted units and the model name.
- ML-09 [E] The ML component shall be callable from the Spring Boot backend through a defined interface (mechanism TBD, D-01).
- ML-10 [E] Training strategy for uploaded data: train per upload on demand, or use a pretrained model (TBD, D-08).
- ML-11 [O] Uncertainty estimates, for example from validation residuals, to size safety stock.
- ML-12 [E] Model limitations shall be documented.

14. Backend and REST API Requirements

- BE-01 [E] Spring Boot REST API exchanging JSON. Indicative endpoints (final in DDS): POST /api/uploads, GET /api/uploads, GET /api/uploads/{id}, POST /api/forecasts, GET /api/forecasts/{id}, POST /api/restock-suggestions, GET /api/template, GET /api/health.
- BE-02 [E] Request validation with HTTP 400 responses and one consistent error format (timestamp, status, message, details).
- BE-03 [E] Layered design with a global exception handler.
- BE-04 [E] Calls to the ML component shall have timeouts and map failures to a clear error response.
- BE-05 [E] Externalised configuration; no hard-coded credentials.
- BE-06 [E] CORS limited to the frontend origin.
- BE-07 [E] Every endpoint testable with Postman or curl and documented.

15. Database Requirements

- DB-01 [E] MySQL 8.x. Indicative tables (final in DDS): uploads, products, sales_records, forecast_runs, forecast_values, restock_suggestions, model_runs.
- DB-02 [E] Primary keys, foreign keys and constraints, including non-negative units and a unique key on (upload, store, product, date).

- DB-03 [E] Index supporting queries by upload, product and date range.
- DB-04 [E] Referential integrity, with cascading delete if FR-20 is built.
- DB-05 [E] The actual schema shall match the approved ER diagram; a sample-data script is provided for testing.
- DB-06 [E] Database credentials kept out of source control.

16. Frontend Requirements

- FE-01 [E] Pages: Upload, Forecast and Suggestion, History, and Help/Template.
- FE-02 [E] Client-side checks (file type and size, numeric horizon, lead time and stock) in addition to server checks.
- FE-03 [E] Loading indicators, error messages and empty states.
- FE-04 [E] Forecast line chart and table; chart library TBD.
- FE-05 [E] Responsive layout.
- FE-06 [E] Plain-language explanation of each suggestion, with units shown.
- FE-07 [E] A visible notice when displayed data is synthetic.

17. Security and Privacy Requirements

- SEC-01 [E] Accept only CSV files within the size limit and validate content.
- SEC-02 [E] Use parameterised queries through the persistence layer; no SQL built by string concatenation.
- SEC-03 [E] Keep credentials out of the repository (environment variables or an ignored local config).
- SEC-04 [E] Store only the needed columns, and tell users not to upload personal customer data.
- SEC-05 [E] Data stays on the local machine; no third-party API calls.
- SEC-06 [E] Error responses shall not expose stack traces.
- SEC-07 [O] If login is built: hashed passwords (BCrypt) and per-user data separation.
- SEC-08 [E] Real shop data may be used only with the owner's permission and after anonymisation.

18. Acceptance Criteria

- AC-01 A valid template CSV uploads, and the stored row count equals the number of valid rows in the file (FR-01 to FR-06).

- AC-02 A CSV missing a required column is rejected with a message naming the column, and nothing is stored (FR-03).
- AC-03 Rows with a negative or non-numeric units value, or an unparseable date, are reported and not stored (FR-03).
- AC-04 A forecast for a product with enough history returns exactly H daily values, starting the day after the last data date (FR-10, FR-11).
- AC-05 A product with too little history returns a clear message and no forecast (FR-14).
- AC-06 Restock suggestions match hand-calculated results for at least three test cases, including one where stock exceeds demand and the suggestion is 0 (FR-16).
- AC-07 MAE and RMSE for the candidate and the baselines on the test period are computed from the actual run and displayed, and the project states honestly whether the candidate beats the baselines (ML-05, ML-06).
- AC-08 With the ML component stopped, the API returns the defined error and the UI shows a clear message (FR-19).
- AC-09 History survives an application restart (FR-18).
- AC-10 The UI is usable at 360 px and 1366 px widths without horizontal scrolling (NFR-07).
- AC-11 The model's feature list does not include the Demand column (ML-04).
- AC-12 Following the README on a clean machine, upload, forecast and suggestion work end to end from the browser.

19. Project Limitations

- The main dataset is synthetic, so results will not show real-world accuracy.
- Small shops have short histories, and filling missing days with zeros can bias a series.
- Weather, local events and promotions are mostly not modelled.
- Units sold may understate demand during stock-outs.
- Metrics from the cited papers use different data and definitions and are not comparable with ours.
- No real-time updates; no automatic ordering.

20. Glossary and References

Glossary

- SKU: a unique product identifier.

- Lead time: days between placing an order and receiving stock.
- Safety stock: extra stock held to cover forecast error.
- Baseline: a simple method a model must beat to be worthwhile.
- Data leakage: using information in training that would not exist at prediction time.
- Chronological split: train on earlier dates, test on later dates.
- MAE / RMSE: average absolute error / root of average squared error.
- ARIMA / SARIMA: classical time-series models; SARIMA adds a repeating seasonal pattern.

References

1. A. Azzahra and I. D. Sudirman, 'Forecasting Daily Sales of Coffee Shop Products: A Comparative Study of Moving Average, ARIMA, and SARIMA,' 2025 ICoDSA, IEEE, DOI 10.1109/ICoDSA67155.2025.11157384.
2. M. Cherian, A. Jane, Deepak, G. Sinchana and MD. Modasscer, 'Demand Forecasting System for Small Business Optimization,' 2025 5th CONIT, IEEE, DOI 10.1109/CONIT65521.2025.11167430.
3. Kim and Cho, 2026, MDPI Forecasting, hybrid clustering for retail demand forecasting (full citation: TBD).
4. Kaggle: 'Retail Store Inventory and Demand Forecasting' (author, URL and licence to be confirmed on the dataset page).

Appendix A. Open Decisions

- D-01: How Spring Boot calls the Python ML component. Candidates: a small Python REST service (leading candidate), or launching a Python process. Decide in the DDS.
- D-02: Final candidate models (scikit-learn regressors, and whether to add statsmodels for ARIMA/SARIMA).
- D-03: Safety-stock method and default lead time.
- D-04: Fixed template column names, or a column-mapping screen.
- D-05: Policy for duplicate rows and missing days.
- D-06: Minimum history length per product.
- D-07: Whether to collect real shop data.
- D-08: Train per upload on demand, or use a pretrained model.
- D-09: Chart library, build tool, operating system and team roles.
- D-10: Horizon options and default, file size limit, maximum products per upload.
- D-11: Whether login is included.

- D-12: Dataset facts to verify: row count, date range, missing values, duplicates, licence.